

GUIÓN DE EXPOSICIÓN

Sistema de Control de Inventario para Ferretería

Grupo #12 | Ingeniería de Software I | Ms. Carlota Delgado Vera

Duración: ~10 minutos | Henry Pazmiño (Slides 1-6 + 13) · Cindy Ayoví (Slides 7-12)

SLIDE 1 — Portada (Henry ~ 30 seg)

Buenos días, profesora y compañeros. Somos el Grupo 12 de Ingeniería de Software I.

El día de hoy vamos a presentarles nuestro proyecto: Sistema de Control de Inventario para Ferretería, una plataforma web diseñada para automatizar la gestión de productos, ventas y stock en tiempo real.

Yo soy Henry Pazmiño, y junto con mi compañera Cindy Ayoví, les explicaremos el análisis, diseño y desarrollo de esta propuesta.

SLIDE 2 — Problema & Oportunidad (Henry ~ 1 min)

Empecemos con el problema que identificamos.

Actualmente, muchas ferreterías en Ecuador administran su inventario de forma manual, usando hojas de cálculo o incluso papel y lápiz. Esto trae cuatro problemas graves:

Primero, control manual deficiente: manejar cientos de productos distintos —con sus respectivos SKU (Stock Keeping Unit, o código de identificación)— en hojas sueltas es propenso a errores.

Segundo, errores de stock constantes: se pierden productos, se vende algo que no hay en bodega, y se generan pérdidas financieras.

Tercero, reportes lentos: consolidar las ventas del día o saber el estado de la bodega toma horas, cuando debería tomar segundos.

Y cuarto, falta de trazabilidad: no hay forma de saber quién registró una entrada o salida de mercadería.

Frente a esto, nuestra propuesta es clara: un sistema web centralizado que resuelve todos estos problemas en una sola plataforma, accesible desde cualquier lugar, las 24 horas del día.

SLIDE 3 — Propuesta de Valor (Henry ~ 45 seg)

¿Qué ofrece exactamente nuestro sistema? Cinco beneficios concretos:

Primero, automatización en tiempo real: cada vez que se registra una venta o un ingreso, el stock se actualiza al instante. Nada de esperar al final del día.

Segundo, alertas de stock mínimo: el sistema avisa automáticamente cuando un producto está por debajo del límite, para que no se queden sin existencias.

Tercero, reportes detallados: se pueden exportar informes de inventario, ventas y movimientos en múltiples formatos.

Cuarto, seguridad por roles: no es lo mismo un administrador que un vendedor. Cada uno tiene acceso solo a lo que necesita.

Y quinto, diseño responsive: la interfaz se adapta a computadores, tablets y celulares.

SLIDE 4 — Análisis de Requerimientos (Henry ~ 1 min 15 seg)

Pasemos al análisis. Identificamos diez requerimientos funcionales (RF) y cuatro no funcionales (RNF).

Del lado izquierdo, los funcionales: empezando por el RF1, Login y autenticación. Luego el RF2, que es el ABM (Altas, Bajas y Modificaciones) de productos: crear, editar, eliminar y consultar. El RF3 para categorías, el RF4 para proveedores, el RF5 para entradas y salidas de stock.

El RF6 son las alertas de stock mínimo que mencionaba antes. RF7 es el registro de ventas. RF8 los reportes de inventario. RF9 la gestión de usuarios y roles. Y RF10 el historial de movimientos.

Del lado derecho, los no funcionales: seguridad con contraseñas cifradas, disponibilidad 24/7, usabilidad con interfaz intuitiva y rendimiento con tiempos de respuesta menores a 3 segundos.

SLIDE 5 — Arquitectura & Modelado (Henry ~ 1 min 15 seg)

Para construir el sistema, usamos una arquitectura en tres capas.

La capa de presentación es la interfaz web, desarrollada con HTML5, CSS3, JavaScript y Bootstrap para que se vea bien en cualquier dispositivo.

La lógica de negocio está en Python con el framework Flask, que maneja todas las reglas del sistema: cálculos de stock, validaciones, permisos.

La capa de datos es MySQL, con siete tablas normalizadas que garantizan consistencia y evitan datos repetidos.

En cuanto al modelado UML (Lenguaje Unificado de Modelado), elaboramos ocho casos de uso que cubren las acciones del administrador y del vendedor. También hicimos un diagrama de secuencia para el control de stock, y el modelo entidad-relación con las siete tablas interconectadas.

Todo este modelado nos permitió tener una visión clara del sistema antes de escribir una sola línea de código.

SLIDE 6 — Cronograma (Henry ~ 45 seg)

El proyecto se desarrolló bajo el modelo incremental, en catorce semanas divididas en cuatro incrementos. Cada incremento agrega funcionalidad completa al sistema.

En el primer incremento, de la semana uno a la cuatro, hicimos el análisis: levantamiento de requerimientos, la WBS (Work Breakdown Structure, estructura de desglose del trabajo) y la matriz RACI (Responsible, Accountable, Consulted, Informed).

En el segundo incremento, de la semana cinco a la ocho, el diseño: definimos la arquitectura de tres capas, los diagramas UML y el modelo entidad-relación.

En el tercer incremento, de la semana nueve a la doce, el desarrollo: programamos en Flask con MySQL y creamos los prototipos responsive.

Y en el cuarto y último incremento, semanas trece y catorce, el cierre: pruebas, mitigación de riesgos y documentación final.

===== TRANSICIÓN: HENRY — CINDY =====

Henry: Y ahora, mi compañera Cindy les va a contar sobre las herramientas que usamos, los prototipos y la calidad del proyecto.

SLIDE 7 — Herramientas (Cindy ~ 1 min)

Cindy: Gracias, Henry. Hola a todos, soy Cindy Ayoví.

En esta diapositiva les muestro las herramientas principales que utilizamos.

Para el control de versiones, usamos Git y GitHub. Trabajamos con un repositorio que tiene seis commits, dos ramas paralelas y dos Pull Requests que fusionamos exitosamente. Esto nos permitió trabajar de forma colaborativa sin pisarnos el código.

Para el desarrollo, usamos Python y MySQL. El backend con Flask nos dio ligereza y velocidad, y MySQL nos aseguró una base de datos relacional sólida y consistente.

Y para la gestión del proyecto, usamos Trello con un tablero Kanban dividido en Tareas, En Proceso y Hecho. También usamos Draw.io para los diagramas UML y los prototipos de las pantallas.

SLIDE 8 — Módulos del Prototipo (Cindy ~ 30 seg)

El prototipo web que diseñamos tiene cinco módulos o pantallas principales.

En las siguientes diapositivas las vamos a ver en detalle, pero les adelanto que tenemos pantalla de Login, Dashboard con métricas, gestión de productos, control de stock y reportes. Cada una pensada para cubrir los requerimientos funcionales que mencionó Henry.

SLIDE 9 — Prototipos 1, 2 y 3 (Cindy ~ 45 seg)

Acá vemos las primeras tres pantallas.

A la izquierda, el Login: un formulario de inicio de sesión seguro, con validación de usuarios y contraseñas cifradas. Solo los usuarios registrados pueden acceder.

Al centro, el Dashboard: un panel de control que muestra métricas en tiempo real, como el total de productos, el stock bajo, las ventas del día y las alertas activas.

A la derecha, la pantalla de Productos: un ABM completo, es decir, permite crear, editar, eliminar y consultar productos. Incluye una tabla filtrable con búsqueda.

SLIDE 10 — Prototipos 4 y 5 (Cindy ~ 30 seg)

Continuamos con las dos pantallas restantes.

A la izquierda, el Control de Stock: aquí se registran todas las entradas y salidas de mercadería. Tiene alertas visuales que cambian de color cuando un producto está por debajo del mínimo.

A la derecha, la pantalla de Reportes: permite exportar reportes dinámicos en PDF y Excel, para que el dueño de la ferretería pueda analizar las ventas y el inventario cuando lo necesite.

SLIDE 11 — Calidad & Riesgos (Cindy ~ 1 min)

Hablemos de calidad. Evaluamos el sistema con cuatro criterios.

Funcionalidad: cumplimos el 100% de los diez requerimientos funcionales. Cada RF tiene su módulo correspondiente funcionando.

Usabilidad: la navegación es clara, gracias a Bootstrap que hace que la interfaz se adapte a cualquier pantalla.

Confiabilidad: las transacciones en MySQL son consistentes. No se pierden datos.

Mantenibilidad: el código es modular y está comentado, así que cualquier programador puede darle mantenimiento después.

En cuanto a riesgos, identificamos cuatro y los mitigamos: primero, cambios en los requisitos — los controlamos con un protocolo estricto de cambios. Segundo, sobrecarga de trabajo — la matriz RACI nos ayudó a distribuir equitativamente. Tercero, fallas de integración — hicimos pruebas continuas de backend y frontend. Y cuarto, curva de aprendizaje — tuvimos sesiones de capacitación previa.

SLIDE 12 — Conclusiones (Cindy ~ 1 min)

Para cerrar esta parte, tres conclusiones y dos recomendaciones.

Primera conclusión: Git y GitHub nos permitieron un flujo de trabajo profesional, con historial trazable y Pull Requests revisados. Aprendimos a trabajar como se trabaja en la industria.

Segunda: aplicar rigor en Ingeniería de Software desde el análisis hasta el prototipado redujo drásticamente el riesgo de tener que rediseñar todo al final.

Y tercera: el sistema cumple con los objetivos planteados y está listo para una implementación real.

Como recomendaciones: primero, integrar un módulo de notificaciones por correo electrónico para alertas automáticas de reabastecimiento. Y segundo, implementar pruebas automatizadas con Flask para validar futuras iteraciones.

===== TRANSICIÓN: CINDY – HENRY =====

Cindy: Y con esto, le devuelvo la palabra a Henry para las preguntas.

SLIDE 13 — Preguntas (Henry ~ 30 seg)

Henry: Muchas gracias, Cindy.

Bueno, profesora y compañeros, eso es todo sobre nuestro proyecto Sistema de Control de Inventario para Ferretería. Hemos visto desde el problema inicial, pasando por el análisis, el diseño, el desarrollo y los prototipos, hasta llegar a las conclusiones.

Estamos listos para responder cualquier pregunta que tengan. Muchas gracias por su atención.